



1η Εργασία

Ονοματεπώνυμο: Παπαθανασίου Ελένη

A.M.: 1115202200135

1 Εισαγωγή

Η εργασία υλοποιεί ένα σύστημα επικοινωνίας μεταξύ διεργασιών σε Linux περιβάλλον χρησιμοποιώντας διαμοιραζόμενη μνήμη και σηματοφόρους. Η κάθε διεργασία μπορεί να δημιουργήσει ή να συμμετάσχει σε ένα διάλογο, να στείλει και να λάβει μηνύματα από άλλες διεργασίες και να τερματίσει το διάλογο αν στείλει το μήνυμα TERMINATE.

Ο κώδικας είναι χωρισμένος σε αρχεία για τη διαχείριση του shared memory, της ουράς μηνυμάτων, των διαλόγων και του συγχρονισμού. Τα αρχεία είναι χωρισμένα σε δύο φακέλους: τα .h αρχεία είναι στον φάκελο include και τα .c στο φάκελο src. Υπάρχουν ακόμα ένα Makefile και ένα README αρχείο.

2 Δομή Αρχείων

2.1 Δομές Δεδομένων

Οι βασικές δομές δεδομένων που χρησιμοποιεί το σύστημα ώστε να λειτουργεί σωστά είναι ορισμένες στο αρχείο common.h και είναι οι εξής:

1. **MSG:** Δομή για την αναπαράσταση ενός μηνύματος. Αποτελείται από ένα id μοναδικό για κάθε μήνυμα, το περιεχόμενο του μηνύματος, το pid του αποστολέα, ένα πίνακα με τους αναγνώστες, το πλήθος των αναγνωστών, το πλήθος των συμμετεχόντων (που πρέπει να διαβάσουν το μήνυμα) και ένα flag για τον τερματισμό, που ενεργοποιείται όταν το μήνυμα είναι το TERMINATE.
2. **MSG_queue:** Δομή για την ουρά των μηνυμάτων κάθε διαλόγου. Αποτελείται από ένα πίνακα με τα μηνύματα, ένα δείκτη για το πιο πρόσφατο μήνυμα, ένα δείκτη για το πιο παλιό μήνυμα, το πλήθος των μηνυμάτων κάθε φορά στην ουρά και το επόμενο διαθέσιμο id.
3. **Dialog:** Δομή για την αναπαράσταση ενός διαλόγου. Αποτελείται από το id του διαλόγου, το πλήθος των συμμετεχόντων του, ένα πίνακα με τα pid των συμμετεχόντων, την ουρά μηνυμάτων γι' αυτό το διάλογο, το id του semaphore set και ένα flag, που δείχνει αν ο διάλογος είναι ενεργός ή όχι.

4. **Dialog_list**: Δομή για την καταγραφή όλων των ενεργών διαλόγων. Αποτελείται από ένα πίνακα με τα id των διαλόγων, το πλήθος των ενεργών διαλόγων και το id του semaphore για το συγχρονισμό της λίστας.

2.2 Αρχεία

1. **common.h**: Ορίζει όλες τις δομές δεδομένων, τις σταθερές, τα IPC keys και κάποιες μακροεντολές.
2. **shared_memory.c/.h**: Περιέχει συναρτήσεις που διαχειρίζονται τα τμήματα του shared memory.
3. **synchronization.c/.h**: Περιέχει συναρτήσεις που δημιουργούν και διαχειρίζονται τα semaphore sets.
4. **dialog_control.c/.h**: Περιέχει συναρτήσεις για τη διαχείριση των διαλόγων.
5. **msg_queue.c/.h**: Περιέχει συναρτήσεις για την υλοποίηση της ουράς μηνυμάτων.
6. **main.c**: Εκτελεί τη βασική λειτουργία του προγράμματος και διαχειρίζεται την επικοινωνία με τον user.

3 Εργαλεία Συγχρονισμού

3.1 Σημαφόροι σε κάθε διάλογο

Για κάθε διάλογο δημιουργείται ένα set από 3 σημαφόρους.

- **sem_access_dialog**: Για την πρόσβαση στη δομή Dialog.
- **sem_empty_slots**: Μετρά τα κενά slot στην ουρά μηνυμάτων.
- **sem_full_slots**: Μετρά τα γεμάτα slot στην ουρά μηνυμάτων.

3.2 Μακροεντολές

Για περισσότερη ευκολία όρισα στο common.h αρχείο μακροεντολές που διαχειρίζονται το κλείδωμα ή ξεκλείδωμα στο διάλογο, τα κενά slot στην ουρά μηνυμάτων και τα γεμάτα slot στην ουρά μηνυμάτων. Χρησιμοποιώ το SEM_UNDO για να εξασφαλίσω την αυτόματη επαναφορά των σημαφόρων σε περίπτωση τερματισμού διεργασίας.

4 Λειτουργίες Συστήματος

4.1 Εκκίνηση και Είσοδος σε διάλογο

Η κάθε διεργασία συνδέεται στη λίστα των διαλόγων και επιλέγει από ένα μενού αν θέλει:

1. Να δημιουργήσει **νέο διάλογο** με τυχαίο μοναδικό id. Ο διάλογος προστίθεται στη λίστα.
2. Να **συνδεθεί σε έναν ήδη υπάρχοντα** διάλογο εισάγοντας το id του διαλόγου.
3. Να δει τη **λίστα με τους** ενεργούς διαθέσιμους **διαλόγους**.
4. Να κάνει **έξοδο** από το σύστημα.

4.2 Αποστολή Μηνύματος

Το σύστημα περιμένει για ένα κενό slot και κλειδώνει το διάλογο. Έπειτα, αντιγράφει τα μηνύματα στην ουρά, ορίζει τον αποστολέα, το id και ενημερώνει το flag terminated αν χρειάζεται. Θέτει τον αποστολέα ως πρώτο αναγνώστη του μηνύματος, ενημερώνει τους δείκτες της ουράς μηνυμάτων και στέλνει σήμα για νέο μήνυμα.

4.3 Λήψη Μηνύματος

Το σύστημα κλειδώνει το διάλογο και διασχίζει την ουρά μηνυμάτων, ψάχνοντας για μηνύματα που δεν έχουν διαβαστεί. Αν βρει μηνύματα, αντιγράφει το μήνυμα, προσθέτει το pid στο πίνακα των αναγνωστών και αν το μήνυμα έχει διαβαστεί από όλους μετακινεί το tail και στέλνει σήμα για κενό slot.

4.4 Τερματισμός Διαλόγου

Τερματισμός μπορεί να γίνει με 2 τρόπους.

1. **Αποστολή TERMINATE:** Το flag γίνεται 1 και όταν λάβουν όλοι οι συμμετέχοντες το μήνυμα γίνεται κλήση του `clean_all()` για την αποσύνδεση και τον τερματισμό.
2. **Επιλογή Ε από μενού:** Καλείται το `leave_dialog()` που αφαιρεί τους συμμετέχοντες από το διάλογο και μετά ο διάλογος αφαιρείται από τη λίστα και διαγράφεται.

5 Διαχείριση Μνήμης και IPC

5.1 Δημιουργία μοναδικών keys

Για να δημιουργηθούν τα μοναδικά κλειδιά η `generate_key` αθροίζει το `IPC_PRIVATE`, που εξασφαλίζεται η μοναδικότητα, το id του διαλόγου και για το `base_value` ένα από τα:

- `key_list` (για τη λίστα διαλόγων)
- `key_sem` (για τα semaphore)
- `key_dialog` (για το shared memory των διαλόγων)

5.2 Καθαρισμός Συστήματος

Με τις συναρτήσεις `detach_d_list()` και `detach_dialog()` γίνεται αποσύνδεση από το shared memory.

Με τις `destroy_d_list()` και `destroy_dialog()` γίνεται διαγραφή από το shared memory.

Με την `clean_all()` καλούνται οι αποσυνδέσεις.

6 Σχεδιαστικές Επιλογές

6.1 Αναγνώστες ανά μήνυμα

Κάθε MSG μήνυμα περιέχει ένα πίνακα `readers` και ένα μετρητή `readers_c`. Όταν ένας χρήστης διαβάσει το μήνυμα, το pid του προστίθεται στον πίνακα. Η συνάρτηση `msg_read_by_all` συγχρύνει τον πίνακα αναγνωστών με τους τρέχοντες συμμετέχοντες του διαλόγου, για να διασφαλίσει ότι κάθε μήνυμα διαβάζεται μία φορά από κάθε συμμετέχοντα.

6.2 Ουρά Μηνυμάτων

Έχω επιλέξει να χρησιμοποιήσω δείκτες για να είναι πιο εύκολο να μετακινώ τα μηνύματα στην ουρά και να ελέγχω το μέγεθός της.

6.3 Διαχείριση του TERMINATE

Ο τελευταίος συμμετέχων ενός διαλόγου καλεί τις συναρτήσεις για τη διαγραφή του διαλόγου και της λίστας διαλόγων. Με αυτό το τρόπο εξασφαλίζω ότι δε θα γίνει terminate πριν έχουν διαβάσει όλοι οι συμμετέχοντες τα μηνύματα.

7 Οδηγίες Εκτέλεσης

Με το **make all** γίνεται μεταγλώττιση των αρχείων.

Με το **make run** γίνεται εκτέλεση σε διαφορετικά terminal.

Με το **make clean** γίνεται καθαρισμός των εκτελέσιμων αρχείων.